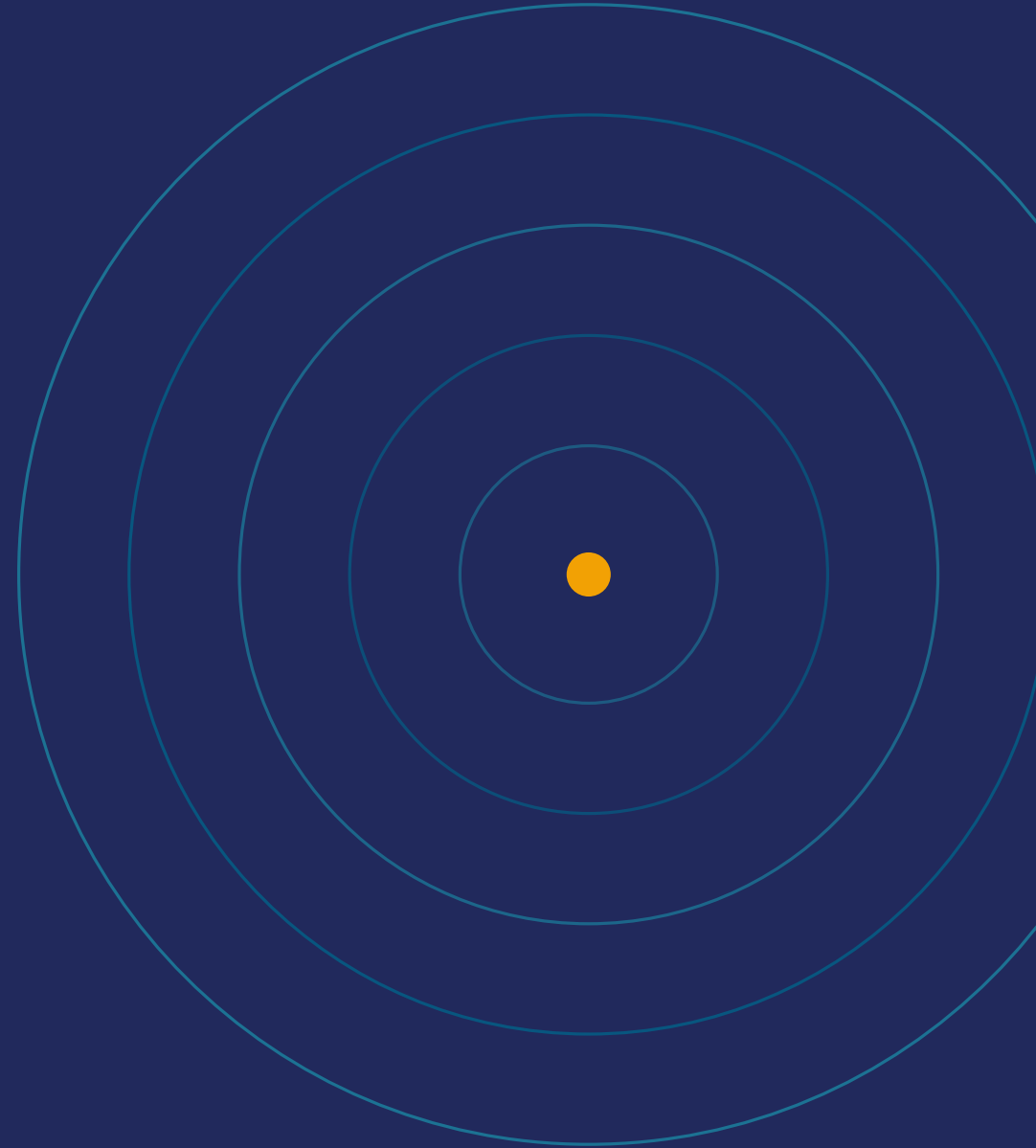


INTRODUCTION TO NEURAL NETWORKS

Radial Basis Function Networks

A local, two-stage alternative to the Multi-Layer Perceptron — built on function approximation and exact interpolation.



Two Ways to Approximate a Function

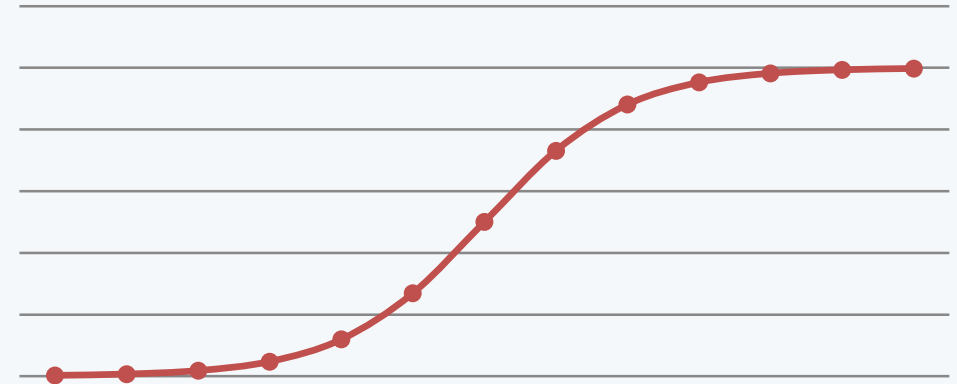
Multi-Layer Perceptron (MLP)

Hidden units use sigmoid activations. Each neuron divides the input space with a hyperplane. Neurons stay active over large, overlapping regions — a **global** approximator. Trained end-to-end with backpropagation.

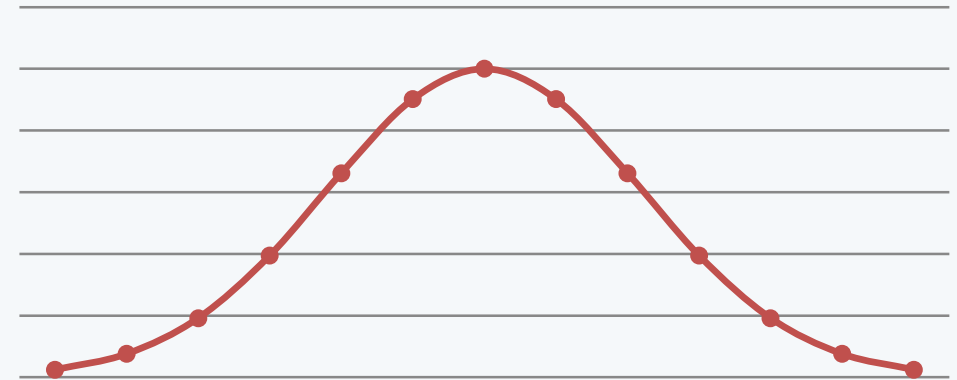
Radial Basis Function (RBF) Network

Hidden units respond only to inputs **near a centre** — a **local** approximator. Trained in two fast, largely non-iterative stages.

MLP hidden unit: sigmoid (global)



RBF hidden unit: Gaussian (local)



Today's focus: RBF networks were built from function-approximation theory — not biology — so we start from interpolation, not neurons.

What Is a Radial Basis Function Network?



Two-layer, feed-forward

Just one hidden layer of basis-function units, plus a linear output layer.



Radial hidden units

Each hidden node implements a radial basis function (e.g. Gaussian) of distance from a centre.



Linear output

Output nodes are simple weighted-sum (linear) units, exactly as in an MLP.



Two-stage training

Input→hidden weights are set first; hidden→output weights are solved second.



Very fast training

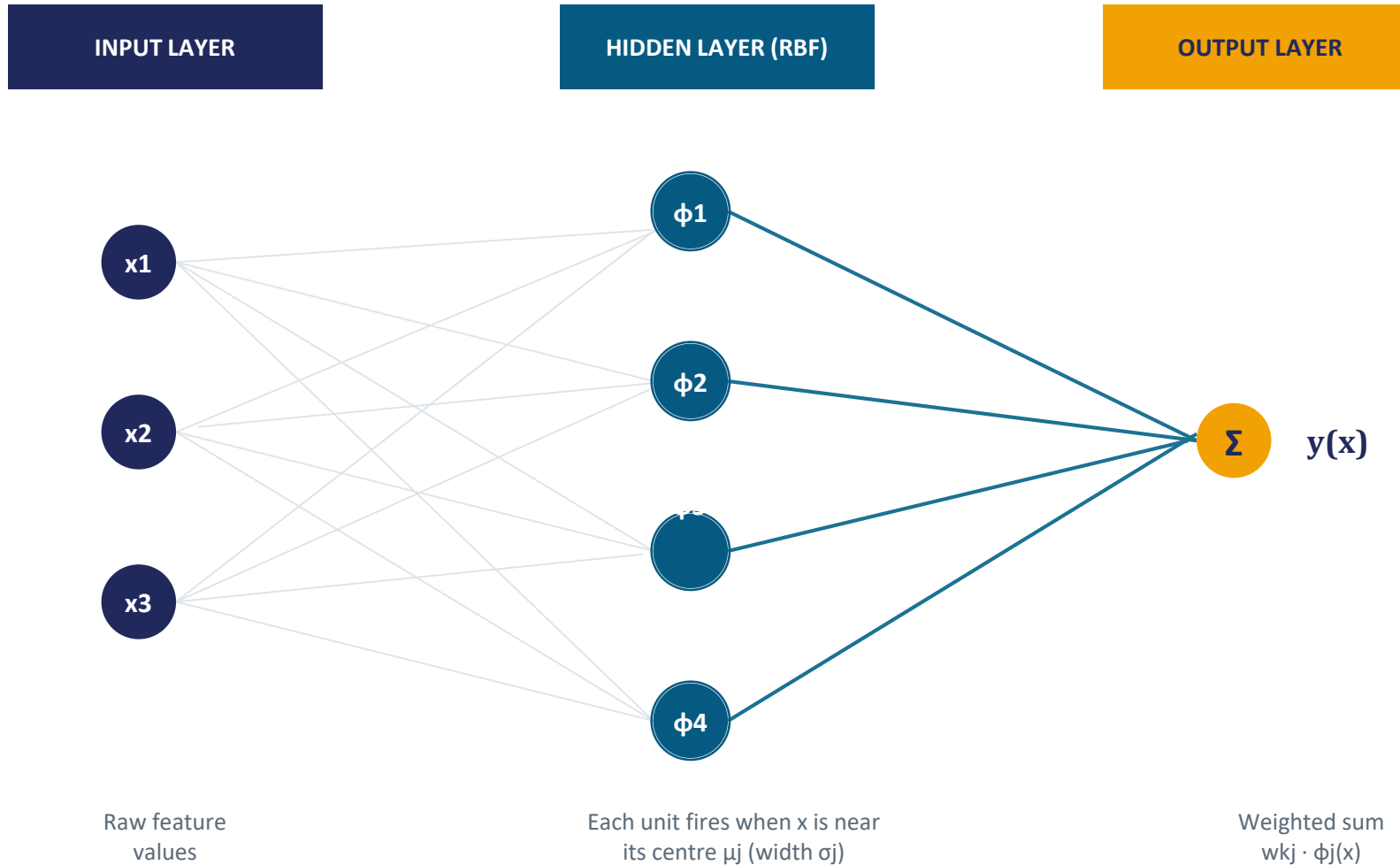
Because the last stage is linear, it can be solved directly — no lengthy gradient descent.



Excellent at interpolation

Built directly from function-approximation / interpolation theory.

Network Architecture at a Glance



Only one hidden layer

Unlike an MLP, depth never grows — RBF nets stay at exactly two computational layers, input→hidden and hidden→output.

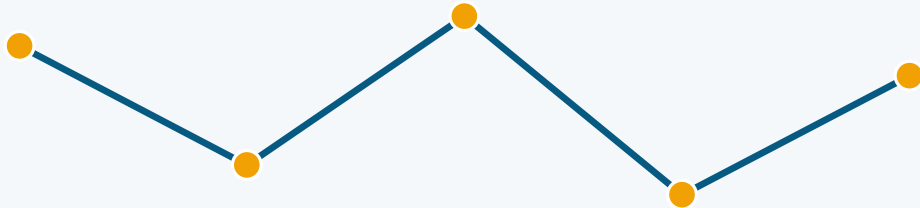
Grey lines: every input reaches every hidden unit.

Teal lines: every hidden unit feeds the output, weighted by w_{kj} .

First, What Does “Interpolation” Mean?

Interpolation is finding a function that reconstructs values *exactly* at every known sample — and estimates sensible values in between. It's the classic approach whenever you have a small set of trustworthy, noise-free measurements.

INTERPOLATION



- ✓ Curve passes through every point exactly
- ✓ Best for sparse, noise-free data
- ✓ This is the RBF network's origin

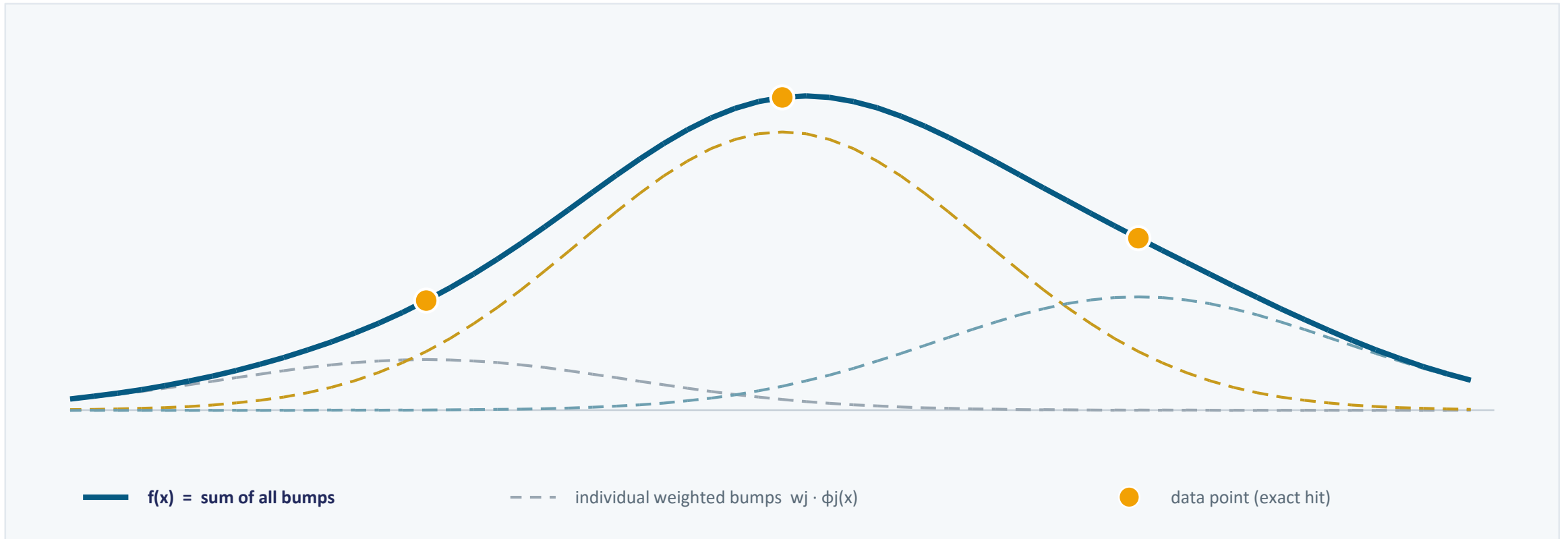
APPROXIMATION / REGRESSION



- ✓ Curve stays close but skips exact points
- ✓ Best for noisy, redundant data
- ✓ This is what backprop-trained MLPs do

How It Works: A Sum of Weighted Bumps

Place one basis function at every data point, scale each by a weight, and add them up. Pick the weights so the sum lands exactly on each target — that's the whole idea.



Formalising Exact Interpolation

Given N data points $(\mathbf{x}^p, t^p)$, the goal is a function $f(\mathbf{x})$ satisfying:

$$f(\mathbf{x}^p) = t^p \text{ for } p = 1, \dots, N$$

RBF places one basis function on every data point, then blends them with a linear combination — exactly what you just saw on the previous slide:

$$f(\mathbf{x}) = \sum_n w_p \cdot \phi(\|\mathbf{x} - \mathbf{x}^p\|)$$

$\phi(\cdot)$ a chosen non-linear function of distance

w_p the weight on the p -th basis function — solved so f hits every target exactly



Why this framing matters

- One basis function per data point means N unknown weights and N equations — a square, solvable system.
- The function is a sum, so it's linear in the weights — even though ϕ itself is non-linear.
- That linearity is what makes the next step — solving for the weights — a simple matrix inversion instead of an iterative search.

Exact Interpolation

The *exact interpolation* of a set of N data points in a multi-dimensional space requires every one of the D dimensional input vectors $\mathbf{x}^p = \{x_i^p : i = 1, \dots, D\}$ to be mapped onto the corresponding target output t^p . The goal is to find a function $f(\mathbf{x})$ such that

$$f(\mathbf{x}^p) = t^p \quad \forall p = 1, \dots, N$$

The radial basis function approach introduces a set of N *basis functions*, one for each data point, which take the form $\phi(\|\mathbf{x} - \mathbf{x}^p\|)$ where $\phi(\cdot)$ is some non-linear function whose form will be discussed shortly. Thus the p th such function depends on the distance $\|\mathbf{x} - \mathbf{x}^p\|$, usually taken to be Euclidean, between $\mathbf{x}$ and $\mathbf{x}^p$. The output of the mapping is then taken to be a linear combination of the basis functions, i.e.

$$f(\mathbf{x}) = \sum_{p=1}^N w_p \phi(\|\mathbf{x} - \mathbf{x}^p\|)$$



The idea is to find the “weights” w_p such that the function goes through the data points.

Solving for the Weights

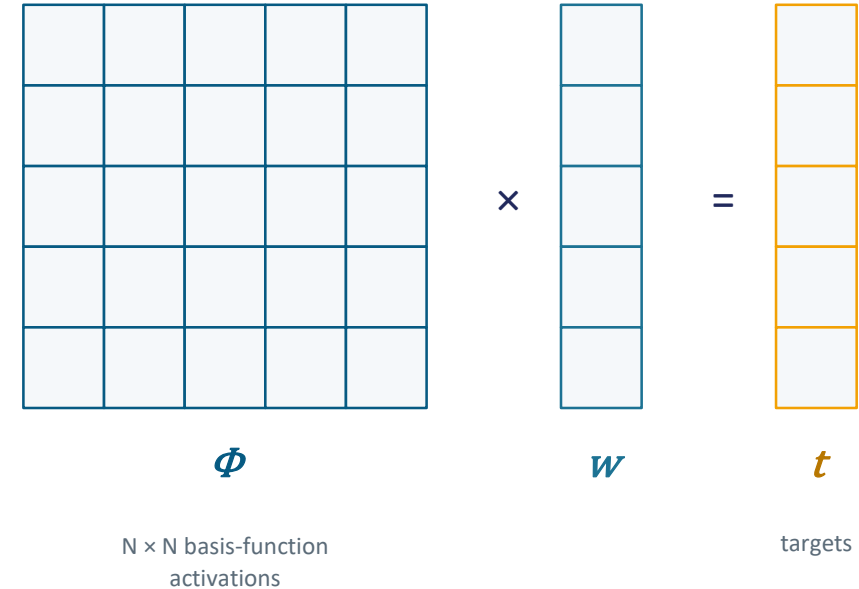
Substituting every training point into $f(x)$ gives one linear equation per data point:

$$\sum q \varphi(\|x^p - x^t\|) = t^p$$

In matrix form, with $\Phi_{pq} = \phi(\|x_q - x_p\|)$:

$$\Phi w = t \quad w = \Phi^{-1} t$$

- ✓ Φ is provably non-singular (invertible) for many common ϕ , as long as the data points are distinct.
- ✓ Once w is known, $f(x)$ is a smooth, continuous, differentiable surface passing exactly through every point.



Determining the Weights

It is easy to determine equations for the weights by combining the above equations:

$$f(\mathbf{x}^q) = \sum_{p=1}^N w_p \phi(\|\mathbf{x}^q - \mathbf{x}^p\|) = t^q$$

We can write this in matrix form by defining the vectors $\mathbf{t} = \{t^p\}$ and $\mathbf{w} = \{w_p\}$, and the matrix $\Phi = \{\Phi_{pq} = \phi(\|\mathbf{x}^q - \mathbf{x}^p\|)\}$. This simplifies the equation to $\Phi \mathbf{w} = \mathbf{t}$. Then, provided the inverse of Φ exists, we can use any standard matrix inversion techniques to give

$$\mathbf{w} = \Phi^{-1} \mathbf{t}$$

It can be shown that, for a large class of basis functions $\phi(\cdot)$, the matrix Φ is indeed non-singular (and hence invertible) providing the data points are distinct.

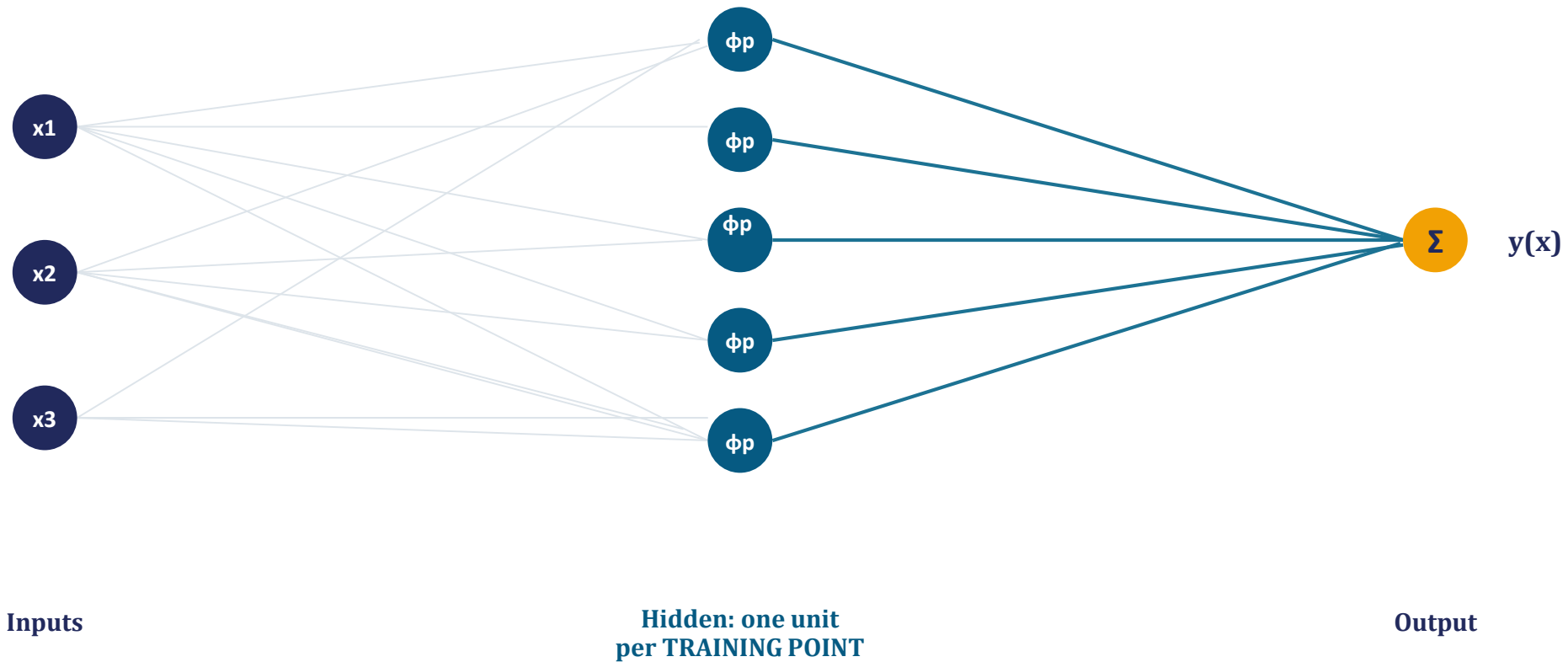
Once we have the weights, we have a function $f(\mathbf{x})$ that represents a continuous differentiable surface that passes exactly through each data point.

This Is Also a Neural Network

Redraw the interpolation equation as a network, and the resemblance to an ANN is exact — just with unusual weights:

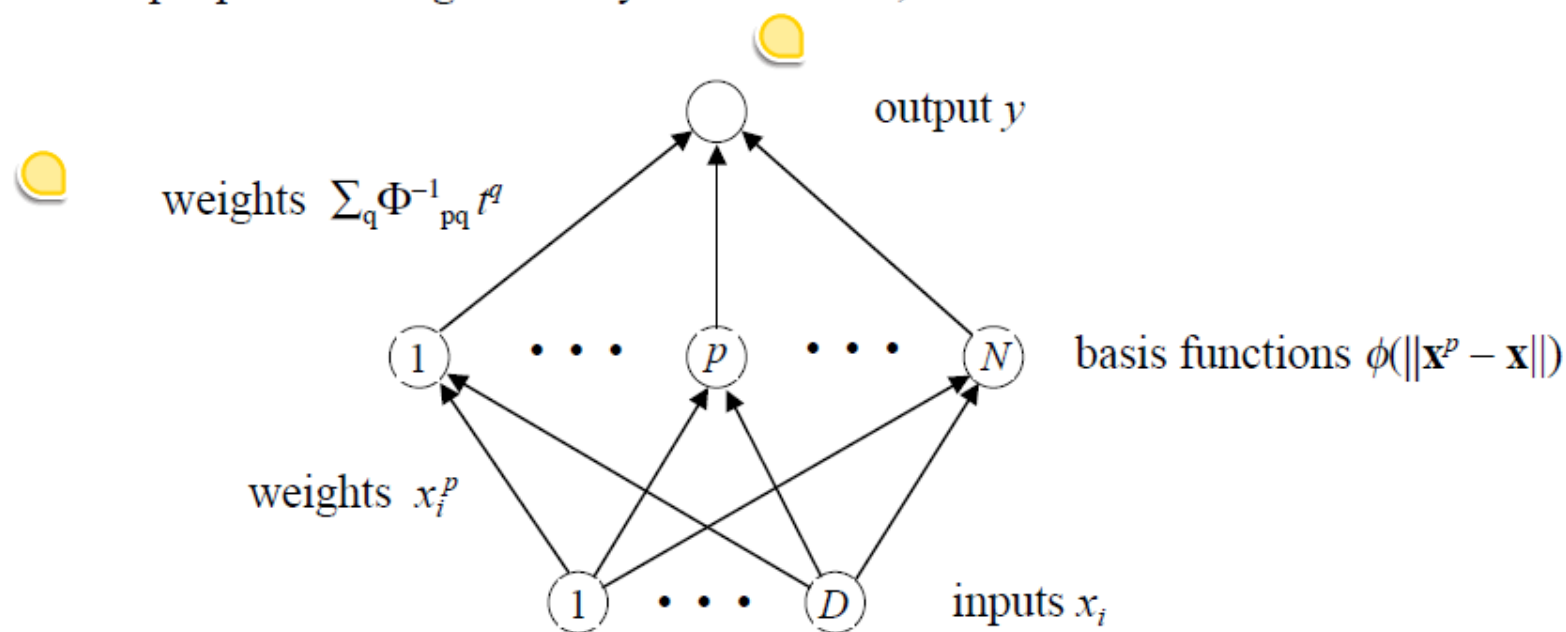
Input $\rightarrow$ hidden “weights” are simply the training inputs x^p themselves — nothing is learned here.

Hidden $\rightarrow$ output weights are the solved values $\sum q \Phi^{-1} p q$ — computed once from Φ^{-1} and the targets.



Radial Basis Function Networks

You might think that what we have just described isn't really a neural network. And a lot of people would agree with you! However, we can see how to make it look like one:



Note that the N training patterns $\{x_i^p, t^p\}$ determine the weights directly. The hidden layer to output weights multiply the hidden unit activations in the conventional manner, but the input to hidden layer weights are used in a very different fashion.

Commonly Used Radial Basis Functions $\varphi(r)$

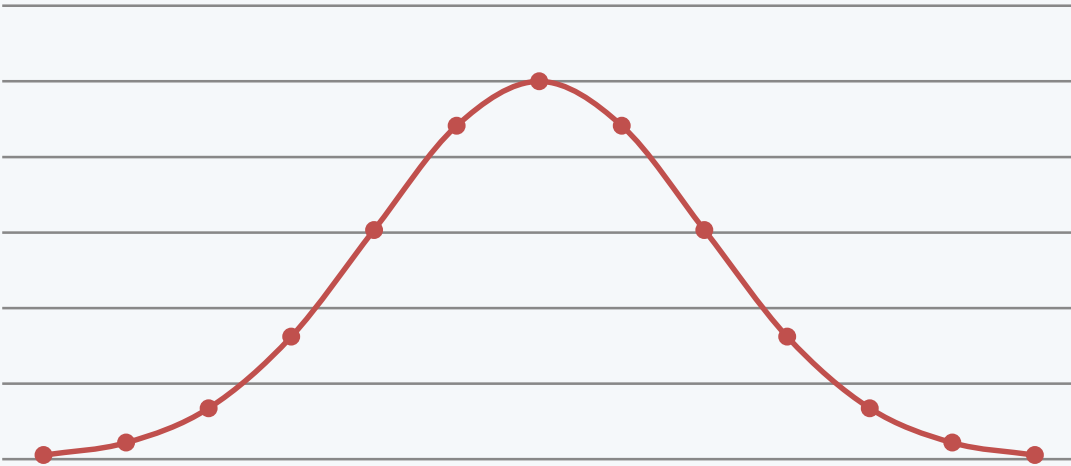
Empirically, the exact shape of φ matters less than you'd expect — but locality (below) matters a lot.

Function	Formula	Behaviour
Gaussian	$\varphi(r) = \exp(-r^2 / 2\sigma^2)$	Localised — the standard choice
Multi-quadric	$\varphi(r) = (r^2 + \sigma^2)^{1/2}$	Grows with r
Inverse multi-quadric	$\varphi(r) = (r^2 + \sigma^2)^{-1/2}$	Localised
Thin plate spline	$\varphi(r) = r^2 \ln(r)$	Grows with r
Cubic	$\varphi(r) = r^3$	Grows with r
Linear	$\varphi(r) = r$	Piecewise-linear interpolation

We'll focus on the Gaussian from here on — it's localised and has convenient analytic properties.

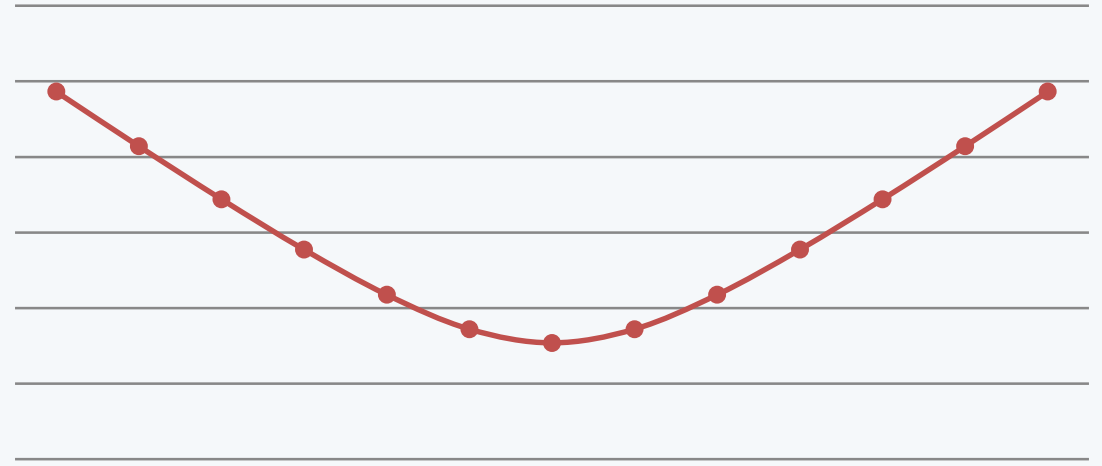
Localised vs. Global Basis Functions

Localised: $\phi(r) \rightarrow 0$ as $r \rightarrow \infty$



e.g. Gaussian, Inverse multi-quadric — activation fades to zero far from the centre.

Unbounded: $\phi(r) \rightarrow \infty$ as $r \rightarrow \infty$



e.g. Multi-quadric, Thin plate spline, Cubic, Linear — still non-linear in x , but never fades out.



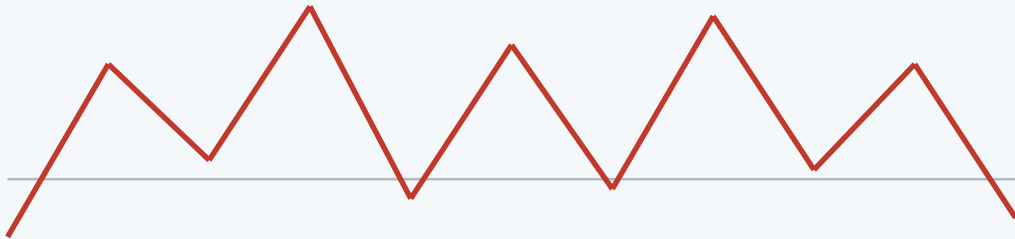
Why locality matters for a neural network: a localised unit only “fires” for inputs near its centre. That gives cleaner, more interpretable feature detectors — which is why Gaussian RBFs are the default in practice.

Two Problems With Exact Interpolation



1. Performs poorly with noisy data

Forcing the curve through every noisy point produces a highly oscillatory function — the same generalisation problem seen with over-fit MLPs.



2. Not computationally efficient

One hidden unit is needed per training pattern. Unlike MLPs — where more data generally helps — here more data means a bigger, slower network.

N training points $\rightarrow$ **N hidden units**

Grows linearly — costly for large datasets.

Improving RBF Networks: Four Changes

1

Fewer hidden units

M basis functions need not equal N data points — in practice $M \ll N$.

2

Learned centres

Centres μ_j no longer have to sit exactly on training inputs; a training algorithm places them.

3

Learned widths

Widths σ_j can vary per unit and are also determined by training — not fixed in advance.

4

Output bias terms

Bias parameters w_{k0} absorb the gap between average basis activation and average target.



Trade-off

Every one of these changes buys efficiency and better generalisation — but it also gives up the clean, exact, closed-form guarantees of the original interpolation network.

*Result: centres, widths, AND output weights
all now need to be learned from data —
which is exactly what the next lecture covers.*

RBF Network vs. Multi-Layer Perceptron

Aspect	MLP	RBF Network
Hidden layers	One or more	Exactly one
Hidden activation	Sigmoid / tanh (global)	Gaussian (local)
Decision regions	Formed by hyperplanes	Formed by hyperspheres
Training	All weights via backprop, jointly	Two separate stages
Typical training speed	Slower (iterative, thousands of epochs)	Much faster (closed-form output stage)
Strength	Global function approximation, extrapolates reasonably	Fast, accurate local interpolation
Weakness	Slower to train	Poor extrapolation; curse of dimensionality

Where RBF Networks Are Used



Function approximation

Modelling smooth continuous relationships from sparse samples.



Pattern classification

Local receptive fields act as natural class-region detectors.



Time-series prediction

Local interpolation captures nearby dynamics well.



Control systems

Fast training suits online / adaptive control settings.

Quick Summary

- RBF networks come from interpolation theory, not biology.
- Exact interpolation (one unit per point) is elegant but noisy and costly.
- Fewer, learned centres and widths make it practical — at the cost of exactness.
- Two-stage training (unsupervised centres, then linear output) makes RBF nets fast to train.



Next Lecture

Determining μ_j , σ_j , and w_{kj} — How RBF Networks Actually Learn



Suggested reading

Bishop — Sections 5.1–5.4

Haykin — Sections 5.1–5.4

Gurney — Section 10.4

Callan — Section 2.6

Hertz, Krogh & Palmer — Section 9.7